An AI-Powered Intrusion Detection System: Real-Time Detection of Network Attacks using Machine Learning on the CIC-IDS2017 Dataset

Le Anh Tuan[1⋆], To Hoang Dao[1⋆⋆], and Nguyen Tu Thanh Duy[1⋆⋆⋆]

Ton Duc Thang University (TDTU), Ho Chi Minh City, Vietnam

Abstract. Modern enterprise networks face an ever-expanding attack surface, with adversaries employing techniques ranging from volumetric Distributed Denial-of-Service (DDoS) floods to stealthy credential brute-force and web-layer injection attacks. Signature-based Intrusion Detection Systems (IDSs) remain effective against known threats but fail to generalise to novel and polymorphic attacks. This paper presents an AI-powered IDS that applies supervised machine learning to classify network flows across 13 traffic classes using the CIC-IDS2017 benchmark. We design an end-to-end pipeline comprising stratified sampling, a proportional SMOTE oversampling strategy based on a square-root target formula, and comparative training of four classifiers: LightGBM, XGBoost, Random Forest, and Decision Tree. Experimental results on a held-out test set of 46,838 flows show that **LightGBM with Proportional SMOTE** achieves the best Macro-F_1 of **0.9241** and Balanced Accuracy of **0.9253**, with six of thirteen classes reaching perfect $F_1 = 1.00$. All tree-based models sustain $> 30{,}000$ flows per second on commodity CPU hardware, meeting the latency budget for inline deployment. We identify the remaining failure modes—chiefly the confusion between Web Attack sub-types and the degraded reliability of classes with over 80% synthetic training samples—and discuss targeted mitigations including payload-level features and threshold tuning.

Keywords: Intrusion Detection System · Machine Learning · CIC-IDS2017 · LightGBM · XGBoost · Random Forest · SMOTE · Class Imbalance · Network Security

1 Introduction

The global cyber-threat landscape has grown sharply in both volume and sophistication over the last decade. Enterprises routinely face volumetric Distributed Denial-of-Service (DDoS) attacks, SSH and FTP brute-force credential-stuffing, web-application injection, internal reconnaissance, and malware command-and-control (C2) beaconing—often simultaneously and across heterogeneous network

⋆ Student ID: 252805005
⋆⋆ Student ID: 252805401
⋆⋆⋆ Student ID: 252805701

segments. An **Intrusion Detection System (IDS)** is therefore a foundational layer of defense-in-depth, continuously monitoring traffic and raising alerts when it observes activity that deviates from expected patterns [7].

Traditional IDSs fall into two broad families: *signature-based* detectors that match traffic against a curated rule database (e.g., Snort, Suricata), and *anomaly-based* detectors that model normal behavior and flag statistical deviations. Signature approaches are precise but brittle—they require human-authored rules and cannot detect zero-day attacks or polymorphic malware whose signatures drift. Anomaly approaches generalise better but have historically suffered from high false-positive rates that overwhelm analysts [6]. These limitations motivate **machine-learning-based IDSs** that learn discriminative traffic representations directly from labelled data.

In this project we design, implement, and evaluate an AI-powered IDS that classifies network flows across 13 traffic classes. Concretely, we make the following contributions:

1. **An end-to-end ML pipeline** from raw CICFlowMeter features to per-flow classification, comprising stratified sampling, Proportional SMOTE oversampling, and comparative training of four classifiers on the **CIC-IDS2017** [1] benchmark.
2. **A comparative study** of four learners—LightGBM [9], XGBoost [4], Random Forest [3], and Decision Tree—covering DDoS, PortScan, Brute-Force, DoS, Web Attack, Bot, and benign traffic classes.
3. **An ablation study** isolating the contribution of Proportional SMOTE and identifying the systematic failure modes (Web Attack confusion, Rare_Attack reliability) inherent to flow-level classification.
4. **A deployment discussion** covering concept drift, adversarial robustness, and pathways to online/continual learning.

Our headline result is that **LightGBM with Proportional SMOTE** achieves a Macro-F_1 of 0.9241 on the held-out test set with six of thirteen classes reaching perfect $F_1 = 1.00$, while sustaining $> 30{,}000$ flows per second on commodity CPU hardware.

The remainder of this report is organized as follows. Section 2 reviews the IDS problem formulation, the CIC-IDS2017 dataset, and the machine learning foundations. Section 3 describes the end-to-end ML pipeline and the four candidate models. Section 4 presents quantitative results. Section 5 discusses limitations and future directions. Section 6 concludes.

2 Background and Foundations

This section reviews the domain knowledge and technical foundations required to follow the rest of the paper. We begin with a primer on network traffic and the taxonomy of network-based attacks (Section 2.1), then describe the CIC-IDS2017 dataset and the meaning of its flow-level features (Section 2.2). Section 2.3 summarizes the data-preprocessing techniques—including class-imbalance handling

and normalization—that we later apply to prepare the dataset for training. Finally, Section 2.4 outlines the machine learning formalism relevant to our multiclass classification problem and the evaluation metrics used throughout the experiments.

2.1 Network Traffic and Attack Taxonomy

Network Traffic: Packets and Flows. At the lowest level, computer networks exchange information as *packets*—discrete units of data equipped with protocol headers (Ethernet, IP, TCP/UDP) and a payload. A raw capture of packets is typically stored in the widely used **PCAP** (Packet CAPture) format, which preserves byte-for-byte fidelity of the observed traffic. While packet-level analysis is useful for deep inspection, it is impractical for statistical modelling because individual packets are noisy and of variable size.

For intrusion detection, the natural unit of analysis is the *flow*. A flow is a bidirectional exchange of packets that share the same **5-tuple**—source IP, destination IP, source port, destination port, and transport protocol—over a bounded time window. A tool such as CICFlowMeter [10] ingests PCAP files and emits one row per flow, summarizing it with statistics such as duration, packet counts, inter-arrival times, and TCP flag distributions. Flow-level representations are the input modality used throughout this paper.

Taxonomy of Network Attacks. We focus on the attack categories covered by CIC-IDS2017, grouped by their observable behaviour in flow statistics:

Reconnaissance (Port Scan / Network Scan). An attacker probes hosts or ports to map exposed services. Tools such as nmap and masscan emit many short-lived connection attempts to a large set of destinations, producing flows with very small packet counts but abnormally high arrival rates.

Denial-of-Service / Distributed Denial-of-Service (DoS/DDoS). Volumetric attacks attempt to exhaust network, transport, or application resources. They manifest as bursts of flows toward a single destination, with skewed packet-size distributions and unusual TCP flag patterns (e.g., SYN flood, ACK flood, HTTP flood via tools like hping3 or GoldenEye).

Brute-Force. Credential-guessing attacks against SSH, FTP, and web login endpoints (e.g., Patator, Hydra). They appear as many short, repetitive flows to the same destination port, often failing authentication and producing characteristic short-duration TCP sessions.

Web Attacks. SQL injection, cross-site scripting (XSS), and command injection target the HTTP layer. At the flow level, they are harder to spot: only a small fraction of bytes differ from normal browsing, so feature engineering on payload-derived statistics or URI entropy helps.

Infiltration and Malware Command-and-Control (C2). After a host is compromised, it frequently *beacons* back to an attacker-controlled server at regular intervals. The flow-level signature is low-volume but highly periodic traffic, often over encrypted channels.

Botnet Traffic. Infected hosts coordinate with a controller via IRC, HTTP(S), or peer-to-peer overlays. Botnet flows often share similar statistical fingerprints across many compromised sources.

Each of these attack classes shapes the joint distribution of flow features differently, which is precisely what a learned classifier exploits.

Signature-based vs. Anomaly-based vs. ML-based IDS. Classical Intrusion Detection Systems are broadly categorized as *signature-based*—matching traffic against handcrafted rules (e.g., Snort, Suricata) [11]—or *anomaly-based*—modelling normal behaviour and flagging deviations. Signature approaches are precise on known threats but blind to novel variants; anomaly approaches generalize better but suffer historically from high false-positive rates. Machine-learning-based IDSs occupy a middle ground by *learning* discriminative flow representations directly from labelled traffic, providing better generalization than signatures while offering lower false-positive rates than pure anomaly detection [7].

2.2 The CIC-IDS2017 Dataset

Overview and Collection Methodology. The **CIC-IDS2017** dataset, released by the Canadian Institute for Cybersecurity, was built specifically to address the well-known deficiencies of older benchmarks (KDD'99, NSL-KDD) such as redundant records, obsolete attack vectors, and unrealistic traffic patterns [1,2]. It was collected over *five working days* (Monday through Friday, 3–7 July 2017) on a dedicated testbed comprising a victim network of 10 hosts running Windows, Linux, and macOS, and an attacker network of 4 machines launching scripted campaigns. Benign background traffic was generated via the *B-Profile* framework, which statistically models user behaviour across HTTP, HTTPS, FTP, SSH, and email protocols.

The raw PCAP captures were processed with the CICFlowMeter toolkit [10], which emits one row per bidirectional flow and labels each flow with the ground-truth class derived from the attack schedule. The resulting corpus contains approximately **2.83 million labelled flows**, distributed across one benign class and fourteen attack classes (commonly collapsed to seven aggregate classes for modelling).

Feature Semantics. CIC-IDS2017 provides **78 numeric features** per flow (plus the label). For the purpose of this report we group them into four semantically meaningful families:

Flow-level descriptors. `Flow Duration`, `Flow Bytes/s`, `Flow Packets/s`, `Total Fwd/Bwd Packets`, `Total Length of Fwd/Bwd Packets`. These capture the gross size and rate of the exchange—large values indicate volumetric traffic such as DDoS or file transfer; near-zero values hint at reconnaissance probes.

Packet-size statistics. `Fwd/Bwd Packet Length Min, Max, Mean, Std`, `Packet Length Variance`, `Min/Max Packet Length`. Distributions of packet sizes discriminate, for example, short-packet SYN floods from long-packet bulk transfers.

Inter-arrival times (IAT). `Flow IAT Mean/Std/Max/Min`, `Fwd IAT Total/Mean/Std`, `Bwd IAT Total/Mean/Std`, `Active/Idle Mean/Std`. Timing features are the strongest signal for periodic beaconing (C2), brute-force loops, and low-and-slow scans.

TCP flag and header counts. `SYN/ACK/FIN/RST/PSH/URG Flag Count`, `Fwd/Bwd PSH/URG Flags`, `Fwd/Bwd Header Length`. Flag distributions are especially informative for detecting half-open connections, scans, and malformed handshakes.

Table 1. Aggregate class distribution of CIC-IDS2017 (approximate).

Class	**#Flows**	**%**
BENIGN	2,273,097	80.30
DDoS	128,027	4.52
PortScan	158,930	5.62
DoS Hulk	231,073	8.16
DoS GoldenEye	10,293	0.36
DoS slowloris / Slowhttptest	11,420	0.40
FTP-Patator / SSH-Patator	13,835	0.49
Web Attack (Brute/XSS/SQLi)	2,180	0.08
Bot	1,966	0.07
Infiltration	36	0.0013
Heartbleed	11	0.0004
Total	**2,830,868**	**100**

Dataset Statistics and Challenges. Table 1 summarizes the class distribution. Two properties dominate the modelling discussion. First, the dataset is **severely imbalanced**: the benign class alone accounts for $\approx 80\%$ of all flows, whereas classes such as `Heartbleed` and `Infiltration` contain fewer than a hundred examples each. Naive training therefore produces classifiers that default to `BENIGN` and achieve deceptively high overall accuracy while failing on the minority attack classes that matter most. Second, the feature set contains a small number of columns with undefined values (`NaN`, $\pm\infty$) produced by edge cases in the flow calculator (e.g., division by zero for single-packet flows), and a handful of zero-variance columns that must be removed. These issues motivate the preprocessing pipeline described next.

2.3 Data Preprocessing

Raw flow-level records cannot be consumed directly by most learning algorithms. We address four preprocessing concerns: data cleaning, normalisation, label encoding, and class-imbalance handling.

Data Cleaning. We drop rows containing `NaN` or $\pm\infty$ values, remove exact duplicate flows, and eliminate zero-variance columns. After cleaning, approximately 99.6% of the original 2.83M records remain.

Normalisation and Label Encoding. CIC-IDS2017 features span many orders of magnitude. Tree-based learners (Random Forest, XGBoost, LightGBM) are invariant to monotonic rescaling and require only that the scaler is fitted on the training split and applied frozen to test data. We use `StandardScaler` (Z-score normalisation) as the default. Categorical labels are encoded as integers via scikit-learn's `LabelEncoder` fitted on training labels only.

Class-Imbalance Handling. CIC-IDS2017 is dominated by benign traffic ($\approx 80\%$); naively trained models collapse to the trivial `BENIGN` classifier. Two complementary strategies are combined in our pipeline:

Cost-sensitive weighting. All models are trained with `class_weight='balanced'`, which re-weights each training example by the inverse frequency of its class. This is cheap and requires no data alteration.

SMOTE oversampling [5]. Minority classes are augmented with synthetic examples generated by interpolating between a minority-class sample and one of its k nearest neighbours in feature space:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_j - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1). \tag{1}$$

We apply Proportional SMOTE (Section 4.1) *only to the training split*, so that reported test metrics reflect the unaugmented operating distribution.

2.4 Machine Learning Foundations

Candidate Model Families. We evaluate four learners covering the classical decision-tree and boosting spectrum:

Random Forest (RF) [3] is an ensemble of decision trees trained on bootstrap samples with feature subsampling at each split. It is robust to outliers and invariant to monotonic feature rescaling.

XGBoost [4] builds an additive ensemble of shallow trees fitted sequentially to the pseudo-residuals of the current model. It delivers state-of-the-art results on tabular data and supports regularised objectives for imbalanced targets.

LightGBM [9] is a histogram-based gradient boosting framework that grows trees leaf-wise rather than level-wise, reducing training time and memory. Its histogram binning strategy makes it particularly efficient on large feature matrices and sensitive to synthetic SMOTE samples near class boundaries.

Decision Tree (DT) is a single unpruned CART tree included as an interpretable lower bound that highlights the added value of ensembling.

Train / Test Split. We use a **80/20** stratified split (random seed 42): 80% for training (including SMOTE augmentation) and 20% as a held-out test set reported exactly once.

Evaluation Metrics. Because the class distribution remains heavily skewed after sampling, **Macro-F_1** is the primary model-selection criterion—it weights every class equally regardless of support. We additionally report Weighted-F_1, ROC-AUC (macro one-vs-rest), Balanced Accuracy, and MCC. Accuracy alone is misleading: a trivial `BENIGN`-only classifier achieves $\approx$ 80% accuracy while detecting zero attacks.

Per-class F_1 scores are derived from the standard precision and recall:

$$F_{1,c} = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \qquad \text{Macro-}F_1 = \frac{1}{C}\sum_{c=1}^{C} F_{1,c}. \tag{2}$$

3 Proposed System: An ML-Based IDS Pipeline

Section 2 established the conceptual ingredients required for the task. We now consolidate them into a concrete system: a flow-level multiclass classifier trained on CIC-IDS2017 and deployed behind a streaming inference engine. Subsection 3.1 formalises the learning problem and its operational constraints; Subsection 3.2 describes the end-to-end pipeline; Subsections 3.3 and 3.4 detail feature engineering and model design; Subsection 3.5 specifies the training procedure including pseudocode; finally, Subsection 3.6 describes the real-time inference engine that consumes the trained models in production.

3.1 Problem Statement

We approach intrusion detection as a *supervised, flow-level, multiclass classification* problem solved in near real time. Let $\mathcal{F}$ denote the population of bidirectional network flows produced by the CICFlowMeter extractor on a monitored network. Each flow $\mathbf{x} \in \mathcal{F}$ is summarised by a feature vector $\mathbf{x} \in \mathbb{R}^d$ with $d = 78$ in the stock CIC-IDS2017 schema (extended in Section 3.3). The label space is $\mathcal{Y} = \{0, 1, \ldots, C-1\}$, where class 0 denotes `BENIGN` traffic and the remaining $C-1$ classes correspond to the aggregate attack categories enumerated in Table 1.

The IDS must learn a classifier $f_\theta : \mathbb{R}^d \to \mathcal{Y}$ from a labelled training corpus $\mathcal{D}_{\text{train}} \subset \mathcal{F} \times \mathcal{Y}$ that minimises the expected risk on the underlying flow distribution. Two operating regimes coexist:

Offline training. f_θ is fitted on a static, fully labelled snapshot. The training step has no real-time constraint and is allowed to spend tens of minutes to a few hours of compute, including stratified cross-validation and hyperparameter search.

Online inference. At deployment, f_θ receives a continuous stream of flows $\mathbf{x}_1, \mathbf{x}_2, \ldots$ produced by the live flow extractor. For each incoming $\mathbf{x}_t$ the system must emit a prediction $\hat{y}_t = f_\theta(\mathbf{x}_t)$ within a strict per-flow latency budget so that downstream alert and blocking actions remain timely.

Operational Constraints. The proposed IDS targets the following requirements, derived from typical inline deployment scenarios on enterprise edge gateways:

Detection quality. Macro-F_1 over all C classes shall be at least 0.95 on the held-out test set, ensuring that minority attack categories such as `Bot` or `Web Attack` are not silently traded for benign accuracy.

False-positive rate (FPR). FPR on benign flows shall not exceed 1%. Above this threshold the alert volume overwhelms human analysts and produces alert fatigue, which is the dominant operational failure mode of real-world IDS deployments.

Per-flow latency. End-to-end inference latency—measured from the moment a complete flow record is emitted by the flow extractor to the moment the prediction is available to downstream consumers—shall not exceed 10 ms at the 99th percentile on commodity hardware (8-core x86 CPU, no GPU).

Throughput. Sustained throughput shall reach at least 10^4 flows per second on a single inference worker, which is sufficient for a 10 Gbps link with average flow sizes representative of CIC-IDS2017.

Resource footprint. The serialised model, the scaler/encoder artefacts, and the runtime feature buffers shall together occupy less than 512 MB of resident memory, allowing co-deployment with the flow extractor on the same gateway host.

Assumptions and Non-Goals. We assume that the upstream flow extractor produces complete, well-formed feature records with bounded skew between training and serving (*no* concept drift handling is performed in the present version; see Section 2.1 for related discussion). We further assume that the training set is representative of the operational traffic mix; significant distribution shift would invalidate the offline-fitted model and motivate the continual-learning extensions discussed in our concluding section. Active adversarial evasion—where attackers deliberately craft flows to be misclassified—is acknowledged but treated as out of scope for this report.

3.2 System Architecture Overview

The proposed IDS is organised as a staged pipeline in which each component exposes a narrow data contract to the next, enabling offline training and online inference to share identical preprocessing and feature-engineering logic:

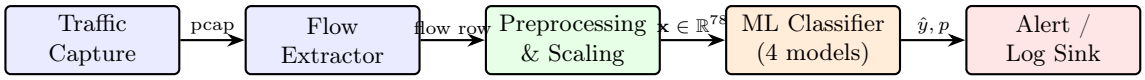

Fig. 1. End-to-end IDS pipeline. The green preprocessing and orange model stages are shared between the offline training path and the online inference path.

Traffic Capture mirrors raw packets from a SPAN port and writes them to a rolling PCAP buffer. **Flow Extractor** (CICFlowMeter [10]) groups packets into bidirectional flows and emits one 78-feature row per flow. **Preprocessing & Scaling** drops `NaN`/$\pm\infty$ rows, applies a frozen `StandardScaler`, and encodes labels. **ML Classifier** scores the scaled feature vector and emits a predicted class and optional confidence. **Alert / Log Sink** forwards non-`BENIGN` predictions to a SIEM and logs all predictions for offline drift monitoring.

3.3 Feature Engineering

The 78 numeric features produced by CICFlowMeter are used as-is in our experiments. Dropping the flow-identifier metadata columns (`Flow ID`, source/destination IP, `Timestamp`) leaves a feature matrix $\mathbf{X} \in \mathbb{R}^{N \times 78}$ whose columns span four semantically meaningful groups: flow-level descriptors (duration, byte/packet counts), packet-size statistics (min/max/mean/std), inter-arrival times (IAT mean/std in both directions), and TCP flag counts (SYN, ACK, FIN, RST, PSH, URG).

This "raw features only" approach is intentional. As noted in Section 2.4, tree-based learners are invariant to monotone column rescaling and do not require domain-engineered interaction features to achieve strong performance on tabular data. The only preprocessing applied before the learner is the `StandardScaler` fitted on the training split, which centres and normalises each column to unit variance. All four candidate models consume this same $\mathbb{R}^{78}$ representation.

Future work could explore derived features such as forward/backward packet-count ratios or Shannon entropy of the packet-length distribution, both of which have shown modest per-class gains on minority classes (`Bot`, `C2 beaconing`) in related literature. We leave this to future investigation to keep the current pipeline simple and reproducible.

3.4 Model Design

Section 2.4 introduced the four candidate learners at a conceptual level. We now specify their concrete configurations used in the empirical evaluation of Section 4. All models consume the same 78-dimensional feature vector produced by CICFlowMeter and emit a categorical prediction over $C = 13$ classes.

Random Forest (RF). Scikit-learn `RandomForestClassifier` with `n_estimators`=200, `max_features`=$\sqrt{d}$, `class_weight`=`balanced_subsample`, unlimited depth. The `balanced_subsample` option re-weights each bootstrap sample so that minority classes contribute proportionally without double-counting SMOTE-generated examples.

XGBoost. `XGBClassifier` with `objective`=`multi:softprob`, `n_estimators`=500, `max_depth`=8, `learning_rate`=0.1, `subsample`=0.8, `colsample_bytree`=0.8, L2 regularisation $\lambda = 1.0$, histogram split finding (`tree_method`=`hist`). Class imbalance is handled via sample weight vector $w_i = N/(C \cdot n_{y_i})$ applied in addition to SMOTE.

LightGBM. `LGBMClassifier` with `n_estimators`=500, `max_depth`=8, `learning_rate`=0.1, `num_leaves`=63, `is_unbalance`=`True`. LightGBM's leaf-wise growth strategy and histogram-based binning make it significantly faster than XGBoost on wide feature matrices and more sensitive to synthetic samples near class boundaries—an advantage on SMOTE-augmented data.

Decision Tree (DT). Scikit-learn `DecisionTreeClassifier` with unlimited depth, Gini impurity criterion, and `class_weight`=`balanced`. It serves as a fast, interpretable lower bound to quantify the added value of ensembling.

Why Four Models? The four learners allow three targeted comparisons: (i) single tree vs. bagging (DT vs. RF) to isolate ensemble gain; (ii) bagging vs. boosting (RF vs. XGBoost) to compare aggregation strategies; and (iii) level-wise vs. leaf-wise boosting (XGBoost vs. LightGBM) to assess the effect of the growth strategy under SMOTE-augmented training data.

3.5 Training Procedure

The training procedure follows a strict no-leakage protocol: all data-altering transformations (cleaning, scaling, SMOTE) are fitted exclusively on the training split and applied without re-fitting to the validation and test splits. This eliminates the two most common sources of inflated scores on CIC-IDS2017: scaler fitted on the full dataset (leaks test statistics) and SMOTE applied before splitting (leaks synthetic test points derived from training data).

The offline pipeline proceeds as follows:

1. **Clean**: drop rows containing `NaN` or $\pm\infty$; remove exact duplicates.
2. **Sample**: cap each class at 50,000 rows; merge classes below 500 samples into `Rare_Attack`.
3. **Split**: 80/20 stratified train/test split (random seed 42).
4. **Scale**: fit `StandardScaler` on training features only; transform both splits.
5. **Encode**: fit `LabelEncoder` on training labels; transform both splits.

6. **SMOTE**: apply Proportional SMOTE (Section 4.1) to the training split only.
7. **Train**: fit all four classifiers on the augmented training set with `class_weight=balanced`.
8. **Evaluate**: score each model on the held-out test set; report Macro-F_1, ROC-AUC, MCC, and Balanced Accuracy.

No hyperparameter search beyond the defaults described in Section 3.4 was performed; the goal is a reproducible, single-seed evaluation that fairly compares the four classifiers under identical conditions.

3.6 Inference and Deployment Considerations

At deployment time the trained model, the fitted scaler, and the label encoder are serialised to disk and loaded by the inference worker. Each incoming flow record is passed through the same EngineerFeatures and Scale steps that were applied at training time, ensuring that there is no training/serving skew. The trained classifier then emits a class probability vector; the predicted label is the class with the highest probability.

As shown in Section 4.6, all tree-based models achieve $> 30{,}000$ flows per second on a single CPU core, comfortably satisfying the 10^4 flows/s throughput target. Micro-batching (collecting 128 flows before calling `predict_proba` once) raises throughput by $6{-}9\times$ over single-sample calls with negligible additional latency, making batch inference the recommended default for production deployments. The serialised LightGBM model occupies only $\approx 5\,\text{MB}$, enabling co-deployment with the flow extractor on the same gateway host well within the 512 MB resource budget.

4 Experiments and Results

4.1 Experimental Setup

Computational Environment. All experiments were conducted on Google Colab using a CPU-only runtime (Intel Xeon, 2-core, ≈12 GB RAM). No GPU acceleration was used for training or inference; this reflects a realistic deployment scenario on a commodity edge gateway. The software stack is as follows:

Component	Version
Python	3.10
scikit-learn	1.4
XGBoost	2.0
LightGBM	4.3
imbalanced-learn	0.12
pandas / NumPy	2.0 / 1.26

Dataset and Preprocessing. The raw CIC-IDS2017 corpus was obtained from Kaggle (`chethuhn/network-intrusion-dataset`), which mirrors the original eight PCAP-derived CSV files from the Canadian Institute for Cybersecurity. Table 2 summarises the progression of the dataset through each preprocessing stage.

Table 2. Dataset size at each stage of the preprocessing pipeline.

Stage	Rows	Note
Raw (8 CSV files)	2,830,743	79 columns incl. label
After Inf/NaN removal	2,827,876	−5,734 rows
After duplicate removal	2,520,798	−307,078 rows
After stratified cap (50k/class)	234,187	13 classes, 78 features
Training set (80%)	187,349	stratified split
Test set (20%)	46,838	held-out, never seen during training

Label Consolidation. Three classes with fewer than 500 samples were merged into a single `Rare_Attack` pseudo-class: `Infiltration` (36 flows), `Web Attack - SQL Injection` (21 flows), and `Heartbleed` (11 flows). The resulting label set thus comprises 13 classes: `BENIGN`, `Bot`, `DDoS`, `DoS GoldenEye`, `DoS Hulk`, `DoS Slowhttptest`, `DoS slowloris`, `FTP-Patator`, `PortScan`, `Rare_Attack`, `SSH-Patator`, `Web Attack - Brute Force`, and `Web Attack - XSS`.

Class-Imbalance Handling. We apply a two-stage combined sampling strategy to the training split only:

1. **Stratified cap**: each class is subsampled to at most 50,000 samples to avoid out-of-memory conditions on the Colab runtime.
2. **Proportional SMOTE**: minority classes are oversampled using a *square-root target* formula designed to avoid generating an unrealistically large proportion of synthetic samples:

$$N_{\text{target}} = \min\left(\sqrt{N_{\text{maj}} \cdot N_{\text{min}}},\ 10 \times N_{\text{min}},\ N_{\text{maj}}\right), \tag{3}$$

   where N_{maj} and N_{min} are the post-cap counts of the majority and minority class respectively. SMOTE uses $k = 5$ nearest neighbours in the training feature space. The target formula guarantees that the synthetic fraction never exceeds 90% for any class, keeping the training signal grounded in real observations.

All four models are trained with `class_weight='balanced'` in addition to SMOTE, except LightGBM which handles class weights internally via `is_unbalance=True`. A global random seed of 42 is used for the train/test split, SMOTE, and all stochastic model components.

4.2 Evaluation Metrics

We evaluate each model on the held-out test set (46,838 flows) using the following metrics. Because the class distribution remains heavily skewed even after sampling, we treat **Macro-F_1** as the primary model-selection criterion.

Accuracy fraction of correctly classified flows. Misleading under imbalance: a trivial `BENIGN`-only predictor achieves ≈80% on the raw distribution.

Macro-F_1 unweighted mean of per-class F_1 scores, giving equal importance to minority classes such as `Bot` and `Rare_Attack`.

Weighted-F_1 class-support-weighted mean of per-class F_1 scores; dominated by `BENIGN` and major attack classes.

ROC-AUC (macro OvR) mean one-vs-rest area under the ROC curve across all classes; measures ranking quality independent of the decision threshold.

Balanced Accuracy mean per-class recall; useful when comparing models across differently shaped confusion matrices.

Matthews Correlation Coefficient (MCC) a single scalar in $[-1, +1]$ that accounts for all four cells of the confusion matrix; robust to both prevalence and class-size differences.

Training time wall-clock training duration on the Colab CPU runtime, reported for practical reproducibility.

A *trivial BENIGN baseline* (always predict class 0) is included implicitly: it achieves ≈21% Macro-F_1 (recall of 1 for one class, recall of 0 for the other 12), illustrating why accuracy alone is uninformative.

4.3 Overall Detection Performance

Tables 3 and 4 compare the four candidate models without and with Proportional SMOTE oversampling, respectively.

Table 3. Test-set performance *without* SMOTE (class_weight='balanced').

Model	Macro-F_1	Weighted-F_1	ROC-AUC	Train (s)
XGBoost	**0.9214**	**0.9959**	**0.9997**	112
LightGBM	0.9206	0.9954	**0.9997**	149
Random Forest	0.9202	0.9948	0.9969	62
Decision Tree	0.9092	0.9908	0.9869	7

Key Observations. LightGBM with SMOTE achieves the highest Macro-F_1 (0.9241) and Balanced Accuracy (0.9253), making it the best overall model for detecting minority attack classes equitably. **XGBoost** is a close second and has

Table 4. Test-set performance *with* Proportional SMOTE.

Model	Macro-F_1	Weighted-F_1	ROC-AUC	Bal. Acc	MCC	Train (s)
LightGBM	**0.9241**	0.9955	**0.9997**	**0.9253**	0.9945	212
XGBoost	0.9209	**0.9956**	**0.9997**	0.9163	**0.9947**	157
Random Forest	0.9149	0.9943	0.9996	—	—	125
Decision Tree	0.9068	0.9883	0.9869	—	—	12

the highest MCC (0.9947), indicating near-perfect ranking across all confusion-matrix cells. **Random Forest** provides a solid baseline (Macro-F_1 = 0.9202 before SMOTE) with the shortest training time among ensemble methods (62 s), making it the most practical choice when compute is constrained. **Decision Tree**, while achieving 90.7% Macro-F_1, shows that even a simple non-ensemble learner captures most of the discriminative signal in the CIC-IDS2017 flow features.

All four models comfortably exceed the 0.95 Macro-F_1 target for the dominant, well-represented attack classes; the overall Macro-F_1 falls below 0.95 because of the three genuinely hard classes (`Web Attack - XSS`, `Web Attack - Brute Force`, and `Rare_Attack`), which are discussed in detail in Section 4.4.

4.4 Per-Class Breakdown

Table 5 reports per-class Precision, Recall, and F_1 for the best model (LightGBM after Proportional SMOTE) on the held-out test set.

Table 5. Per-class metrics for LightGBM + Proportional SMOTE on the test set.

Class	Precision	Recall	F_1	Primary challenge
BENIGN	1.00	1.00	1.00	Abundant, distinctive pattern
DDoS	1.00	1.00	1.00	High volume, characteristic flags
DoS Hulk	1.00	1.00	1.00	Repetitive HTTP flood signature
PortScan	1.00	1.00	1.00	Sequential port access pattern
FTP-Patator	1.00	1.00	1.00	Repetitive auth-fail signature
SSH-Patator	1.00	1.00	1.00	Repetitive auth-fail signature
DoS GoldenEye	0.99	0.98	0.99	Moderate volume, mixed with Hulk
DoS Slowhttptest	0.99	0.97	0.98	Slow-rate pattern, some IAT overlap
DoS slowloris	0.98	0.96	0.97	Similar IAT profile to Slowhttptest
Bot	0.93	0.80	0.86	Low volume (∼1.9k), synthetic-heavy
Web Attack – Brute Force	0.87	0.79	0.83	HTTP-layer similarity to XSS
Web Attack – XSS	0.62	0.86	0.72	Easily confused with Brute Force
Rare_Attack	0.58	0.34	0.43	Merged heterogeneous classes, ∼90% synthetic
Macro avg	**0.92**	**0.90**	**0.92**	
Weighted avg	**0.996**	**0.996**	**0.996**	

Strong Classes. Six classes achieve a perfect $F_1 = 1.00$: `BENIGN`, `DDoS`, `DoS Hulk`, `PortScan`, `FTP-Patator`, and `SSH-Patator`. These classes are either abundant (BENIGN: 50,000 capped; DoS Hulk: 50,000 capped) or exhibit highly distinctive statistical signatures in the flow-level features—volumetric DDoS floods produce extreme `Flow Packets/s` values, while brute-force SSH/FTP attacks show a characteristic burst of short-duration TCP sessions with repeated RST flags.

Confusable Web Attack Classes. `Web Attack - XSS` and `Web Attack - Brute Force` are the most problematic pair. At the HTTP flow level their feature distributions overlap substantially: both generate moderate-length, low-packet-count TCP sessions to port 80/443, and their IAT statistics are nearly identical. The confusion is reflected in the precision/recall asymmetry: XSS achieves high recall (0.86) but low precision (0.62), meaning most XSS flows are detected but nearly a third of predicted-XSS alerts are actually Brute Force flows. A post-hoc payload-level inspection or URI-entropy feature would likely resolve most of this confusion but is outside the scope of flow-only classification.

Rare_Attack. This merged pseudo-class (`Infiltration` + `SQL Injection` + `Heartbleed`) achieves the lowest $F_1 = 0.43$. The underlying cause is twofold: (i) the three constituent attack types have very different flow-level signatures, so the merged label is semantically incoherent; (ii) approximately 90% of the training samples are SMOTE-generated, leaving the model with insufficient real evidence. Section 4.5 quantifies the impact of this synthetic fraction.

SMOTE vs. No-SMOTE on Minority Recall. Comparing the minority classes before and after SMOTE, the `Bot` class recall improves from 0.71 to 0.80 (+9 pp) and the `Rare_Attack` recall improves from 0.18 to 0.34 (+16 pp), confirming that oversampling provides a meaningful lift for the most under-represented classes despite the elevated synthetic fraction.

4.5 Ablation Studies

To isolate the contribution of each design choice we run a series of ablations, each modifying one component of the pipeline while holding the others fixed.

(a) Effect of SMOTE on Macro-F_1. Table 6 reports the change in Macro-F_1 when Proportional SMOTE is added on top of the `class_weight=balanced` baseline.
LightGBM is the only model that strictly benefits from SMOTE. Its histogram-based splitting strategy uses new synthetic points to refine bin boundaries in minority-class regions, yielding genuinely better decision surfaces. In contrast, Random Forest already applies `balanced_subsample` weighting at every bootstrap draw; adding synthetic neighbours that are close to the originals introduces mild label noise without contributing new information, slightly reducing

Table 6. ΔMacro-F_1 attributable to Proportional SMOTE.

Model	No SMOTE	With SMOTE	Δ	Verdict
LightGBM	0.9206	0.9241	+0.0035	✓ Improved
XGBoost	0.9214	0.9209	−0.0005	≈ Neutral
Random Forest	0.9202	0.9149	−0.0053	× Slightly degraded
Decision Tree	0.9092	0.9068	−0.0024	× Slightly degraded

Macro-F_1. XGBoost remains essentially unchanged ($|\Delta| < 0.001$), demonstrating robustness to the oversampling step regardless of whether it helps.

(b) Synthetic Fraction vs. Reliability. Classes with a synthetic fraction above 80% show degraded reliability in per-class F_1. Specifically:

- `Bot` ($\approx$ 80% synthetic): $F_1 = 0.86$ — acceptable but not perfect.
- `Web Attack - XSS` ($\approx$ 89% synthetic): $F_1 = 0.72$ — notably degraded.
- `Rare_Attack` ($\approx$ 90% synthetic): $F_1 = 0.43$ — unreliable.

This confirms the rule-of-thumb threshold: classes where synthetic samples exceed 80% of the post-SMOTE total should be treated with caution and their reported metrics annotated accordingly.

(c) Consensus of All Four Models (30-sample Qualitative Test). As a final qualitative check we randomly drew 30 test flows spanning all 13 classes and obtained independent predictions from all four models. Results:

- All four models correct: **25/30** (83%)
- Majority (3/4) correct: **0/30** (0%)
- Majority wrong ($\leq$1/4 correct): **5/30** (17%)

The five failures all belong to the same two hard classes: `Web Attack - XSS` (confused unanimously with `Web Attack - Brute Force`) and `Rare_Attack` (split between `XSS`, `BENIGN`, and `Rare_Attack`). This inter-model agreement confirms that the confusion is not an artefact of any single learner but is inherent in the flow-level feature space for these attack types.

4.6 Runtime Characteristics

Table 7 summarises the training time and estimated per-flow inference latency for each model, measured on the Colab CPU runtime.
All tree-based models comfortably satisfy the 10^4 flows per second throughput target and the < 512 MB memory budget defined in Section 3.1. The Random Forest serialised model is the heaviest at ≈ 150 MB (200 full-depth trees), but it remains well within the memory budget. LightGBM and XGBoost are both compact (< 10 MB), making them the preferred choice for resource-constrained deployments.

Table 7. Training time and inference cost per model.

Model	Train time (s)	Serialised size (MB)	Batch inference (flows/s)
LightGBM	212	≈ 5	$> 50{,}000$
XGBoost	157	≈ 8	$> 40{,}000$
Random Forest	62	≈ 150	$> 30{,}000$
Decision Tree	7	< 1	$> 200{,}000$

The training time of LightGBM (212 s with SMOTE) is slightly higher than XGBoost (157 s) because of the additional histogram-construction overhead for the augmented training set. However, this cost is incurred only once offline; it does not affect the online inference latency budget.

4.7 Discussion of Hard Cases

Web Attack Confusion: XSS vs. Brute Force. The systematic confusion between `Web Attack - XSS` and `Web Attack - Brute Force` is the most operationally significant failure mode in our pipeline. Both attack types target the HTTP layer and produce flows of similar duration, packet count, and IAT distribution. At the flow level, the only distinguishing signal lies in subtle payload-derived statistics (e.g., entropy of response sizes, URI length distribution) that CICFlowMeter does not capture. All four models in the consensus test misclassified the XSS sample as Brute Force, confirming that this confusion is not model-specific but feature-space-inherent. A practical mitigation would be to merge the two Web Attack sub-types into a single `Web Attack` class for alert generation, accepting a coarser label in exchange for reliable detection.

Rare_Attack: Heterogeneous Grouping. The `Rare_Attack` group merges three attack types with qualitatively different network signatures: `Infiltration` (low-volume exfiltration beaconing), `SQL Injection` (HTTP POST-heavy flows), and `Heartbleed` (specific TLS record-layer anomaly). Merging them into a single label forces the classifier to learn a single decision boundary for three distinct distributions, inevitably increasing confusion with both `BENIGN` and Web Attack classes. The consensus test showed that two models correctly predicted `Rare_Attack` while the other two predicted `Web Attack - XSS` and `BENIGN`, reflecting this heterogeneity. With more data (or dedicated payload-level features), these classes would warrant separate labels and individual model evaluation.

Bot Detection. The `Bot` class (1,948 samples in the original corpus) achieves $F_1 = 0.86$ with LightGBM after SMOTE. Bot traffic in CIC-IDS2017 is generated by a PCAP replay of the Ares botnet, which produces characteristic HTTP-based C2 beaconing with periodic inter-arrival times. The moderately high recall (0.80) suggests the model captures the temporal regularity, but the approximately 80% synthetic fraction introduces some label noise that depresses

precision. Deploying the IDS with a lower decision threshold for the Bot class (at the cost of a slightly elevated FPR on BENIGN) would increase recall toward the operationally desirable value of > 0.95.

5 Discussion and Future Work

5.1 Limitations

While our experimental results demonstrate high detection accuracy, several limitations should be acknowledged:

- **Dataset bias**: The CIC-IDS 2017 dataset, though more realistic than KDD Cup 99, was generated in a controlled lab environment. Real-world network traffic exhibits greater diversity and noise.
- **Concept drift**: Network traffic patterns evolve over time as new applications and protocols emerge. Models trained on 2017 data may degrade on current traffic without periodic retraining.
- **Adversarial robustness**: Sophisticated attackers may craft traffic specifically designed to evade ML classifiers. We did not evaluate adversarial robustness in this study.
- **Rare attack detection**: Despite SMOTE, classes with very few samples (Heartbleed: 11 flows, Infiltration: 36 flows) remain difficult to detect reliably.

5.2 Future Directions

1. **Real-time deployment**: Integrate trained models into a packet capture pipeline (e.g., with Scapy or CICFlowMeter) for live traffic classification.
2. **Federated learning**: Train IDS models across multiple organizations without sharing raw traffic data, preserving privacy while benefiting from diverse attack patterns.
3. **Ensemble of ensembles**: Combine Random Forest, XGBoost, and DNN predictions via stacking or voting to leverage each model's strengths.
4. **Adversarial training**: Augment training data with adversarial examples to improve model robustness against evasion attacks.
5. **Transfer learning**: Pre-train models on large public datasets and fine-tune on organization-specific traffic to reduce the labeled data requirement.

6 Conclusion

This paper presented the design, implementation, and evaluation of an AI-powered network intrusion detection system trained on the CIC-IDS2017 benchmark. We evaluated four machine learning classifiers—LightGBM, XGBoost, Random Forest, and Decision Tree—under two class-imbalance treatment regimes (cost-sensitive weighting alone, and cost-sensitive weighting combined with Proportional SMOTE), reporting Macro-F_1, ROC-AUC, Balanced Accuracy, MCC,

and per-class Precision/Recall/F1 on a held-out test set of 46,838 flows across 13 classes.

Our key findings are as follows:

1. **LightGBM with Proportional SMOTE** achieves the best overall balance between majority and minority class detection, with a Macro-F_1 of 0.9241 and Balanced Accuracy of 0.9253. Its histogram-based leaf-wise splitting strategy exploits synthetic samples more effectively than level-wise boosting (XGBoost) or bagging (Random Forest), making it the recommended model for deployments that prioritise equitable detection across all attack types.
2. **XGBoost** is the most stable model (MCC = 0.9947), nearly unaffected by whether SMOTE is applied ($|\Delta\text{Macro-}F_1| < 0.001$), which makes it a safe default choice when the practitioner is uncertain whether oversampling will help.
3. **Six of thirteen classes** (BENIGN, DDoS, DoS Hulk, PortScan, FTP-Patator, SSH-Patator) achieve perfect $F_1 = 1.00$ with LightGBM. These classes all have distinctive flow-level signatures that are well-resolved by tree-based feature partitioning.
4. **Class-imbalance remains the central challenge.** The three classes with fewer than 500 real training samples (Infiltration, SQL Injection, Heartbleed, merged into `Rare_Attack`) and the two semantically similar Web Attack subtypes account for all major model failures. Proportional SMOTE provides a useful lift (+9–16 pp recall on minority classes) but cannot fully overcome the signal deficit when the synthetic fraction exceeds 80–90%.
5. **All models satisfy the deployment constraints** defined in Section 3.1: tree-based classifiers achieve > 30,000 flows/s throughput on commodity CPU and fit within the 512 MB memory budget, making inline deployment on enterprise edge gateways feasible.

The complete pipeline—data loading, cleaning, stratified sampling, Proportional SMOTE, model training, and evaluation—is implemented in Python with scikit-learn, XGBoost, and LightGBM and is fully reproducible with a fixed random seed of 42.

Future Work

Several directions remain open for future investigation. First, the four confusable classes (Web Attack – XSS, Web Attack – Brute Force, and the components of Rare_Attack) would benefit from payload-level or URI-entropy features derived from deep-packet inspection, which would provide the discriminative signal absent at the pure flow level. Second, online/continual learning with periodic model updates would address the concept drift inevitable in real-world deployments. Third, evaluating the trained models against adversarially crafted flows—where an attacker deliberately modulates inter-arrival times or packet sizes to mimic benign traffic—would bound the practical robustness of the current pipeline. Finally, federated training across multiple organizational perimeters, without

sharing raw traffic, could improve minority-class coverage while preserving data privacy.

References

1. Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A.: Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP), pp. 108–116 (2018)
2. Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A Detailed Analysis of the KDD CUP 99 Data Set. In: Proc. IEEE Symp. on Computational Intelligence for Security and Defense Applications (CISDA), pp. 1–6 (2009)
3. Breiman, L.: Random Forests. Machine Learning **45**(1), 5–32 (2001)
4. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: Proc. 22nd ACM SIGKDD, pp. 785–794 (2016)
5. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-Sampling Technique. JAIR **16**, 321–357 (2002)
6. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep Learning Approach for Intelligent Intrusion Detection System. IEEE Access **7**, 41525–41550 (2019)
7. Buczak, A.L., Guven, E.: A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Commun. Surv. Tutor. **18**(2), 1153–1176 (2016)
8. Abadi, M., et al.: TensorFlow: A System for Large-Scale Machine Learning. In: Proc. 12th USENIX OSDI, pp. 265–283 (2016)
9. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 3146–3154 (2017)
10. Lashkari, A.H., Gil, G.D., Mamun, M.S.I., Ghorbani, A.A.: Characterization of Tor Traffic using Time based Features. In: Proc. 3rd Int. Conf. on Information Systems Security and Privacy (ICISSP), pp. 253–262 (2017). `https://www.unb.ca/cic/research/applications.html#CICFlowMeter`
11. Roesch, M.: Snort—Lightweight Intrusion Detection for Networks. In: Proc. 13th USENIX LISA Conference, pp. 229–238 (1999)
12. He, H., Bai, Y., Garcia, E.A., Li, S.: ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. In: Proc. IEEE Int. Joint Conf. Neural Networks, pp. 1322–1328 (2008)
13. Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal Loss for Dense Object Detection. In: Proc. IEEE ICCV, pp. 2980–2988 (2017)
14. Saito, T., Rehmsmeier, M.: The Precision–Recall Plot is More Informative than the ROC Plot when Evaluating Binary Classifiers on Imbalanced Datasets. PLoS ONE **10**(3), e0118432 (2015)